know that such prevention is basically a five-minute job.

In the remaining part of the article, I will walk you through a tool called Talisman, which can help you to prevent disasters, if you agree to spend just five minutes on it.

The Talisman way

Talisman is an open source tool created by Thoughtworks. It installs a hook to a repository to ensure that potential secrets or sensitive information do not leave the developer's workstation. It validates the outgoing change set for things that look suspicious such as potential SSH keys, authorisation tokens, private keys, etc. It supports MacOSX, Linux and Windows. Talisman can be installed as a pre-commit hook or a pre-push hook. However, I strongly recommend the pre-commit hook, because it will help you to save time on editing your commit message again. Talisman sits on your machine's home (or a parent location of your choice, some place where you keep all your Git repositories) as a Git hook. You can install it once and it will take care of any secrets being accidentally pushed to VCS from your existing or new Git repositories. Once installed, Talisman's auto update mechanism takes care of updating new features to the installation whenever there is a new release. To install Talisman, run the following command on a terminal. This will download and install the binary at *$HOME/.talisman/bin*.

As a pre-commit hook (recommended), give the command:

```
curl --silent  https://raw.
githubusercontent.com/thoughtworks/
talisman/master/global_install_scripts/
install.bash > /tmp/install_talisman.
bash && /bin/bash /tmp/install_
talisman.bash
```

As a pre-push hook, give the command:

```
curl --silent https://raw.githubusercontent.com/thoughtworks/talisman/master/global_
install_scripts/install.bash > /tmp/install_talisman.bash && /bin/bash /tmp/install_
talisman.bash pre-push
```

Follow the simple instructions in your terminal and you are done! Let's look at how it works. After you install Talisman, it will run checks for obvious secrets automatically, whenever a Git's *commit/push* command is invoked (as chosen during installation). In case there are any potential secrets detected, Talisman will display a detailed report of the errors:

```
$ git commit -m "Commit Message"
Talisman Report:
+----------------+-------------------------------------------------------------+
|     FILE       |                            ERRORS                           |
+----------------+-------------------------------------------------------------+
| danger.pem     | The file name "danger.pem"                                  |
|                | failed checks against the                                   |
|                | pattern ^.+\.pem$                                           |
+----------------+-------------------------------------------------------------+
| danger.pem     | Expected file to not to contain hex encoded texts such as:  |
|                | awsSecretKey=c64e8c79aacf5ddb02f1274db2d973f363f4f553ab16   |
+----------------+-------------------------------------------------------------+
```

Talisman CLI support for existing Git repositories

We just figured out how Talisman prevents sensitive information from leaving the developer's or QA's machine and getting checked in to VCS. This works perfectly for any new repositories that you create. Now, what about the secrets that were already checked in to an existing repository? How do you look at the complete Git history and find out secrets which were accidentally checked in before, and remove them? Talisman also supports a CLI tool, which you can run from your repo. This finds out existing secrets in a Git repository. Here's how you can use the Git History Scanner support of the Talisman CLI (you can potentially add it to your CI/CD pipeline to make secret scanning a part of your deployment process).

If you execute the command *talisman* from your repository, you will be presented with the following options.

```
    --c string: Short form of checksum calculator
    --checksum string: Checksum calculator calculates checksum and suggests
.talsimarc format
     --d:  Short form of debug
     --debug: Enable debug mode (warning: very verbose)
     --githook string: Either pre-push or pre-commit (default 'pre-push')
     --p string: Short form of pattern
     --pattern string:  Pattern (glob-like) of files to scan (ignores git hooks)
     --s: Short form of scanner
     --scan: Scanner scans the git commit history for potential secrets
     --scanWithHTML: Scans the repo and generates pretty HTML reports
     --v:  Short form of version
     --version: Shows current version of talisman
```

Follow these steps to perform the scan:

- To get into the Git directory path to be scanned, use the command *cd <directory to scan>*